

Chapter 5, Consensus Algorithms

Outline



- Introducing the consensus problem
- Consensus mechanism analysis and design
- Classification
- Types of algorithm
- Choosing an algorithm

Consensus problem



- Consensus is a fundamental problem in distributed systems.
- Since the 1970s this problem has been researched in the context of distributed systems, but recently, with the advent of blockchain technology, a renewed interest has arisen in developing distributed consensus algorithms that are suitable for blockchain networks.
- Discuss the underlying techniques behind distributed consensus algorithms, their inner workings, and new algorithms that have been specifically developed for blockchain networks

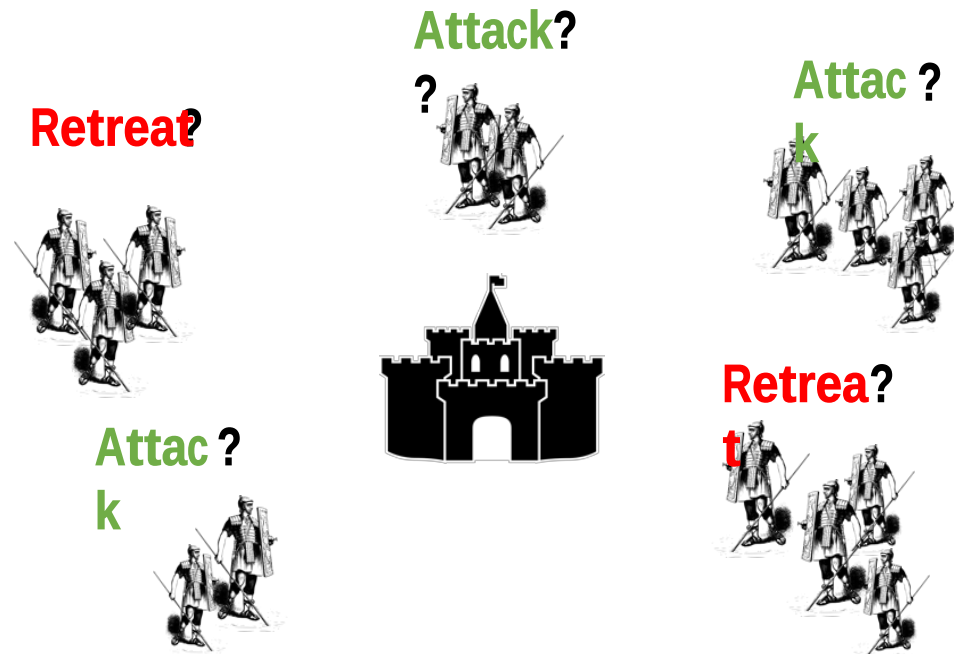
- Distributed systems are classified into two main categories, namely **message passing** and **shared memory**.
- In the context of blockchain, we are concerned with the message passing type of distributed systems, where participants on the network communicate with each other via passing messages to each other
- Blockchain is a distributed system that relies upon a consensus mechanism, which ensures the safety and liveness of the blockchain network.
- Distributed consensus is a highly researched problem and many fundamental ideas to describe and elaborate on the problem have been developed. One of them, and arguably the most famous, is the Byzantine generals problem

The Byzantine generals problem

5

- In distributed systems, a common goal is to achieve consensus (agreement) among nodes on the network even in the presence of faults. In order to explain the problem, Lamport came up with an allegorical representation of the problem and named it the **Byzantine generals problem**.

The famous Byzantine generals problem was formulated by Lamport et al. in their paper: Lamport, L., Shostak, R. and Pease, M., 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems*, 4(3), pp.382-401.



Fault tolerance



- A fundamental requirement in a consensus mechanism is that it must be fault-tolerant.
- In other words, it must be able to tolerate a number of failures in a network and should continue to work even in the presence of faults.
- This naturally means that there has to be some limit to the number of faults a network can handle, since no network can operate correctly if a large majority of its nodes are failing.
- Based on the requirement of fault tolerance, consensus algorithms are also called fault-tolerant algorithms, and there are two types of fault-tolerant algorithms.

- The first is **Crash fault-tolerance (CFT)** and the other is **Byzantine fault-tolerance (BFT)**.
- CFT covers only crash faults or, in other words, benign faults.
- In contrast, BFT deals with the type of faults that are arbitrary and can even be malicious.
- **Replication** is a standard approach to make a system fault-tolerant.
- Replication results in a synchronized copy of data across all nodes in a network.
- This technique improves the fault tolerance and availability of the network.
- This means that even if some of the nodes become faulty, the overall system/network remains available due to the data being available on multiple nodes.

- There are two main types of replication techniques:
- **Active replication**, which is a type where each replica becomes a copy of the original state machine replica.
- **Passive replication**, which is a type where there is only a single copy of the state machine in the system kept by the primary node, and the rest of the nodes/replicas only maintain the state.
- State machine is a mathematical model that is used to describe a machine that can be in different states.
- A state machine can only have one state at a time. A state machine stores a state of the system and transitions it to the next state as a result of input received. As a result of state transition, an output is produced along with an updated state.

State machine replication



- **State machine replication (SMR)** is a de facto technique that is used to provide deterministic replication services in order to achieve fault tolerance in a distributed system.
- State machine replication was first proposed by Lamport in 1978 in his paper
Lamport, L., 1978. Time, clocks, and the ordering of events in a distributed system.
Communications of the ACM, 21(7), pp.558-565.
- The paper is available here:
- <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/Time-Clocks-and-the-Ordering-of-Events-in-a-Distributed-System.pdf>

- The fundamental idea behind SMR can be summarized as follows:
- All servers always start with the same initial state.
- All servers receive requests in a **totally ordered** fashion (sequenced as generated from clients).
- All servers produce the same deterministic output for the same input.
- State machine replication is implemented under a primary/backup paradigm, where a primary node is responsible for receiving and broadcasting client requests. This broadcast mechanism is called **total order broadcast** or **atomic broadcast**, which ensures that backup or replica nodes receive and execute the same requests in the same sequence as the primary.
- Consequently, this means that all replicas will eventually have the same state as the primary, thus resulting in achieving consensus.

- Fault tolerance works up to a certain threshold.
- For example, if a network has a vast majority of constantly failing nodes and communication links, this type of network may not be as fault-tolerant as we might like it to be.
- In other words, even in the presence of fault-tolerant measures, if there is a lack of resources on a network, the network may still not be able to provide the required level of fault tolerance.
- In some scenarios, it might be impossible to provide the required services due to a lack of resources in a system.
- In distributed computing, such impossible scenarios are researched and reported as impossibility results.

- In distributed computing, impossibility results provide an understanding of whether a problem is solvable and the minimum resources required to do so.
- If the problem is unsolvable, then these results give a clear understanding that a specific task cannot be accomplished and no further research is necessary.
- Impossibility results show that certain problems are not computable under insufficient resources.
- The requirement of minimum available resources is known as **lower bound results**.
- The problems that are not solvable under any conditions are known as **unsolvability results**.

FLP impossibility

- FLP impossibility is a fundamental unsolvability result in distributed computing theory that states that in an asynchronous environment, the deterministic consensus is impossible, even if only one process is faulty
- FLP is named after the authors' names, Fischer, Lynch, and Patterson, who in 1985 introduced this result. This result was presented in their paper:
- Fischer, M.J., Lynch, N.A. and Paterson, M.S., 1982. Impossibility of distributed consensus with one faulty process (No. MIT/LCS/TR-282). *Massachusetts Inst of Tech Cambridge lab for Computer Science*.
- The paper is available at: <https://apps.dtic.mil/dtic/tr/fulltext/u2/a132503.pdf>

- To circumvent **FLP impossibility**, several techniques have been introduced in the literature. These techniques include:
- **Failure detectors**, which can be seen as **oracles** associated with processors to detect failures.
- **Randomized algorithms** have been introduced to provide a probabilistic termination guarantee. The core idea behind the randomized protocols is that the processors in such protocols can make a random choice of decision value if the processor does not receive the required quorum of trusted messages.
- **Synchrony assumptions**, where additional synchrony and timing assumptions are made to ensure that the consensus algorithm terminates and makes progress.

Lower bounds on the number of processors to solve consensus

15

- lower bound is the minimum amount of resources, for example, the number of processors or communication links required to solve a problem.
- In other words, if a minimum required number of resources is not available in a system, then the problem cannot be solved.
 - In the case of CFT, at least $2F + 1$ number of nodes is required to achieve consensus.
 - In the case of BFT, at least $3F + 1$ number of nodes is required to achieve consensus.

Where F = number of faults.

Analysis and design

- Designing and analyzing consensus mechanisms requires a model that can be used to study and explore system properties and make designs more logical.
- A computational model represents processes, network conditions, timing assumptions, and how all these entities interact and work together

A model consists of:

- Processes: Processes communicate with each other by passing messages to each other
- Timing assumptions: are made when designing consensus algorithms
 - Synchrony: known upper bound on the communication and processor delays, run on synchronous networks
 - Asynchrony: no upper bound on the communication and processor delays, asynchronous networks without any timing assumptions
 - Partial synchrony: an upper bound on the communication and processor delays, however, this upper bound is not known to the processors

Classification

Since the invention of Bitcoin, which introduced lottery-based consensus algorithms, there are two main types of consensus:

- Traditional
 - This category includes Paxos, BFT algorithms
- Lottery-based
 - This category includes modern Proof of Work (PoW) algorithms

Consensus correctness properties

18

- Safety
 - Agreement
 - Validity
 - Integrity
- Liveness
 - Termination

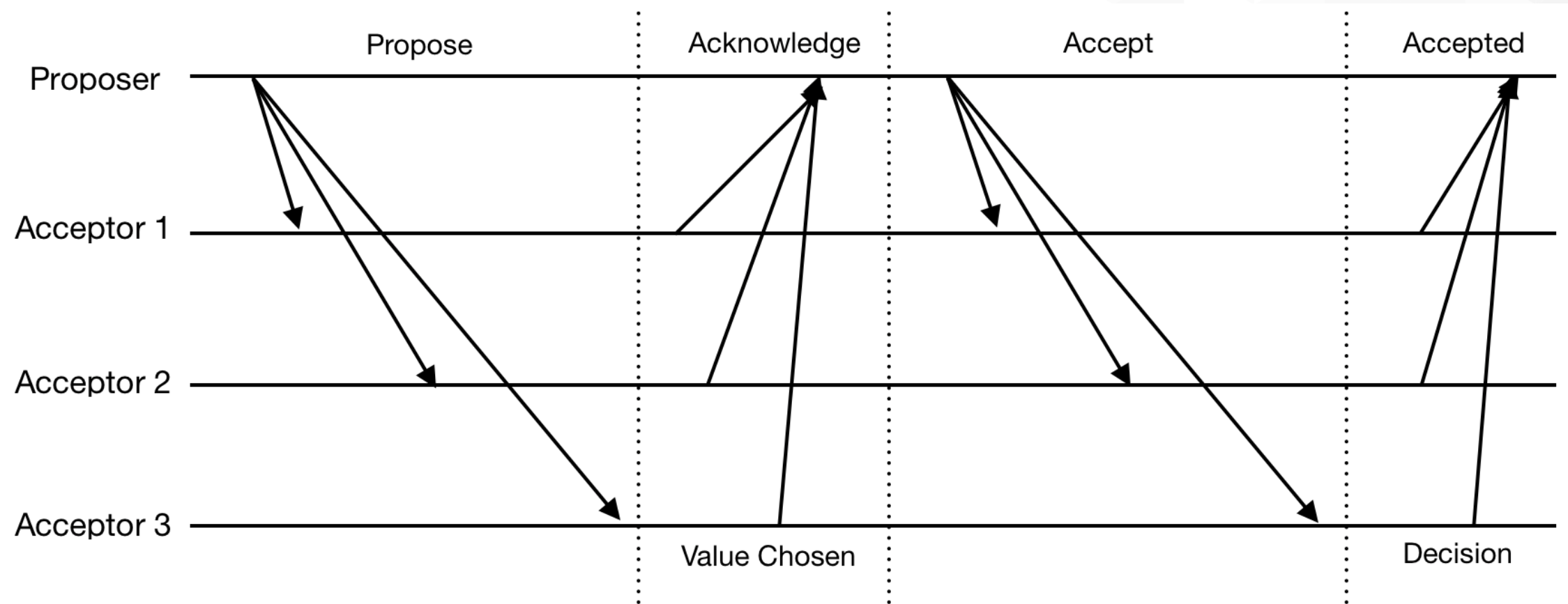
Algorithms

19

- CFT
 - Paxos
 - Raft
- BFT
 - PBFT
 - IBFT
 - Tendermint
 - HotStuff
- Nakamoto and post-Nakamoto
 - PoW
 - Proof of Stake (PoS)

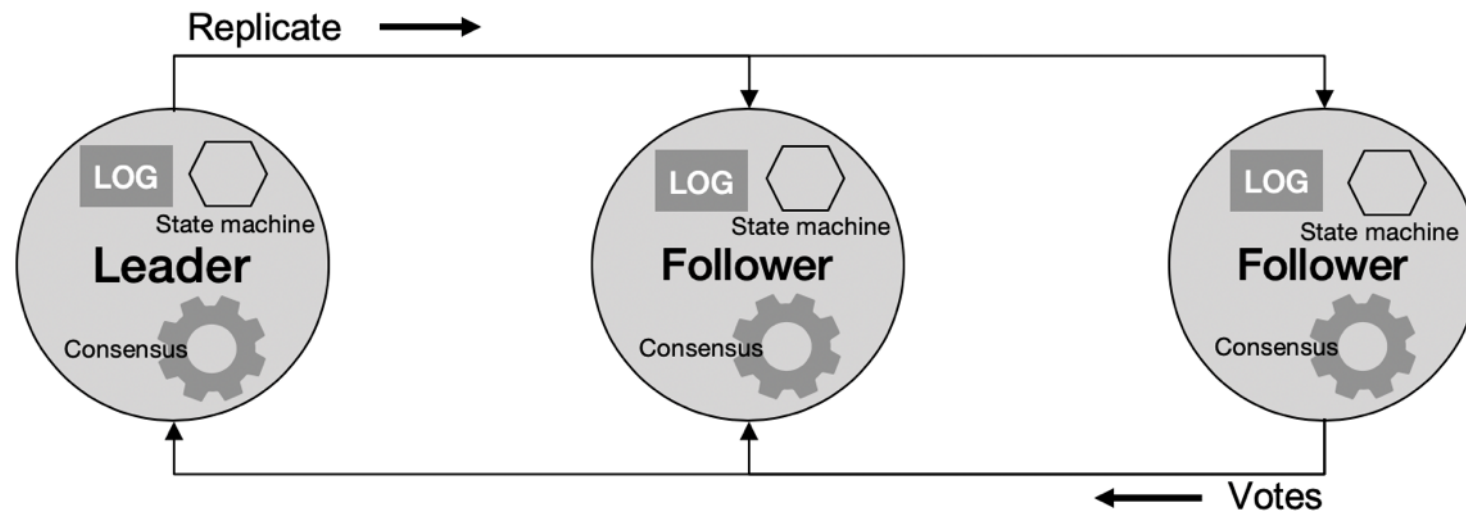
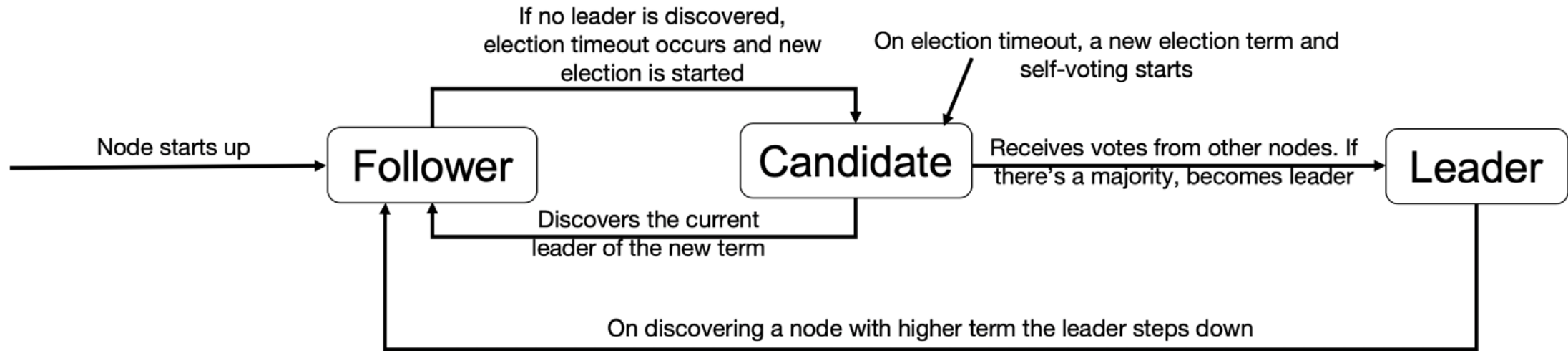
Paxos

20



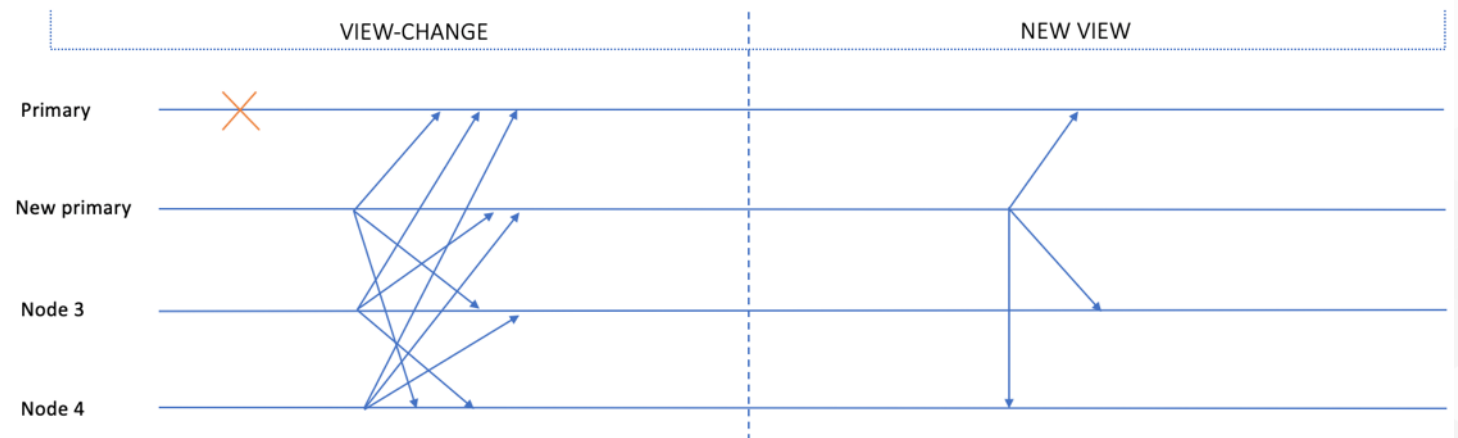
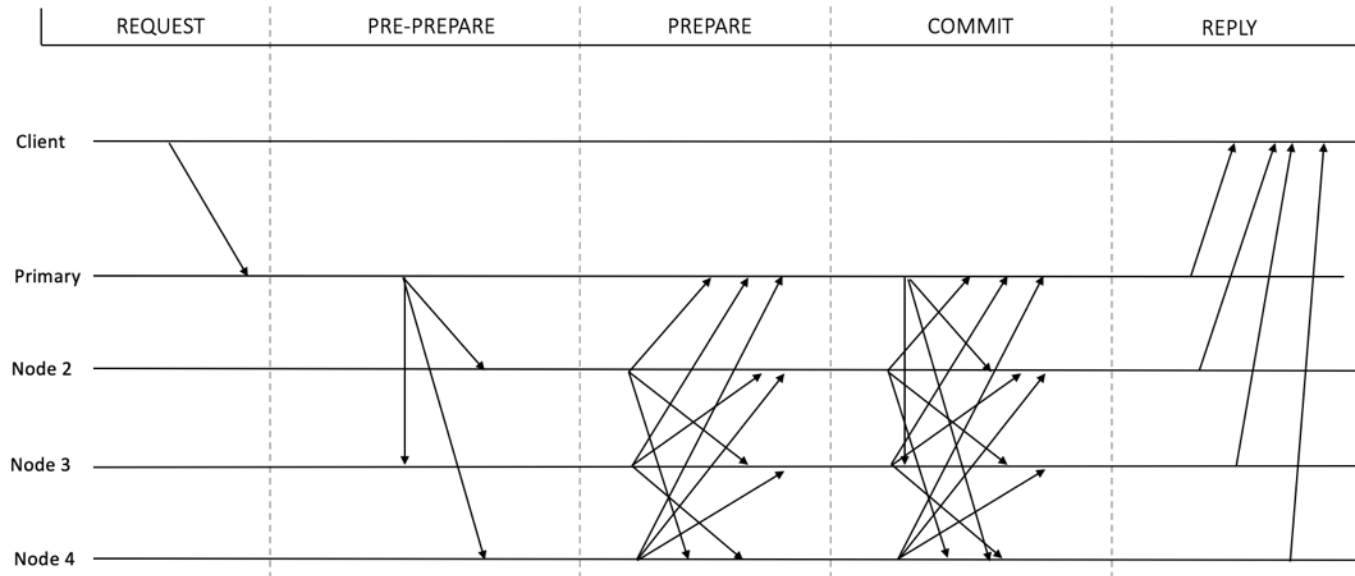
Raft

21



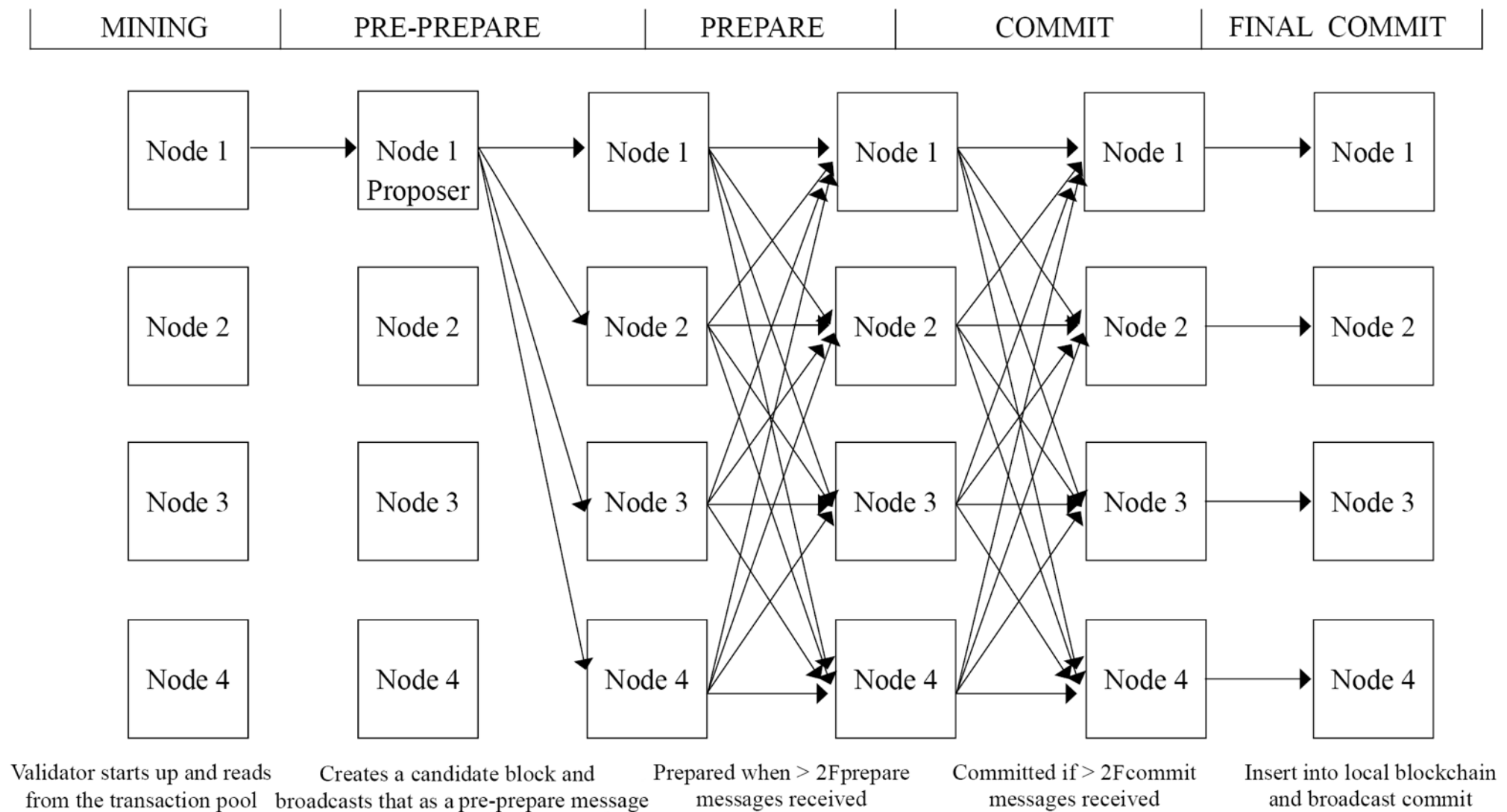
PBFT

22

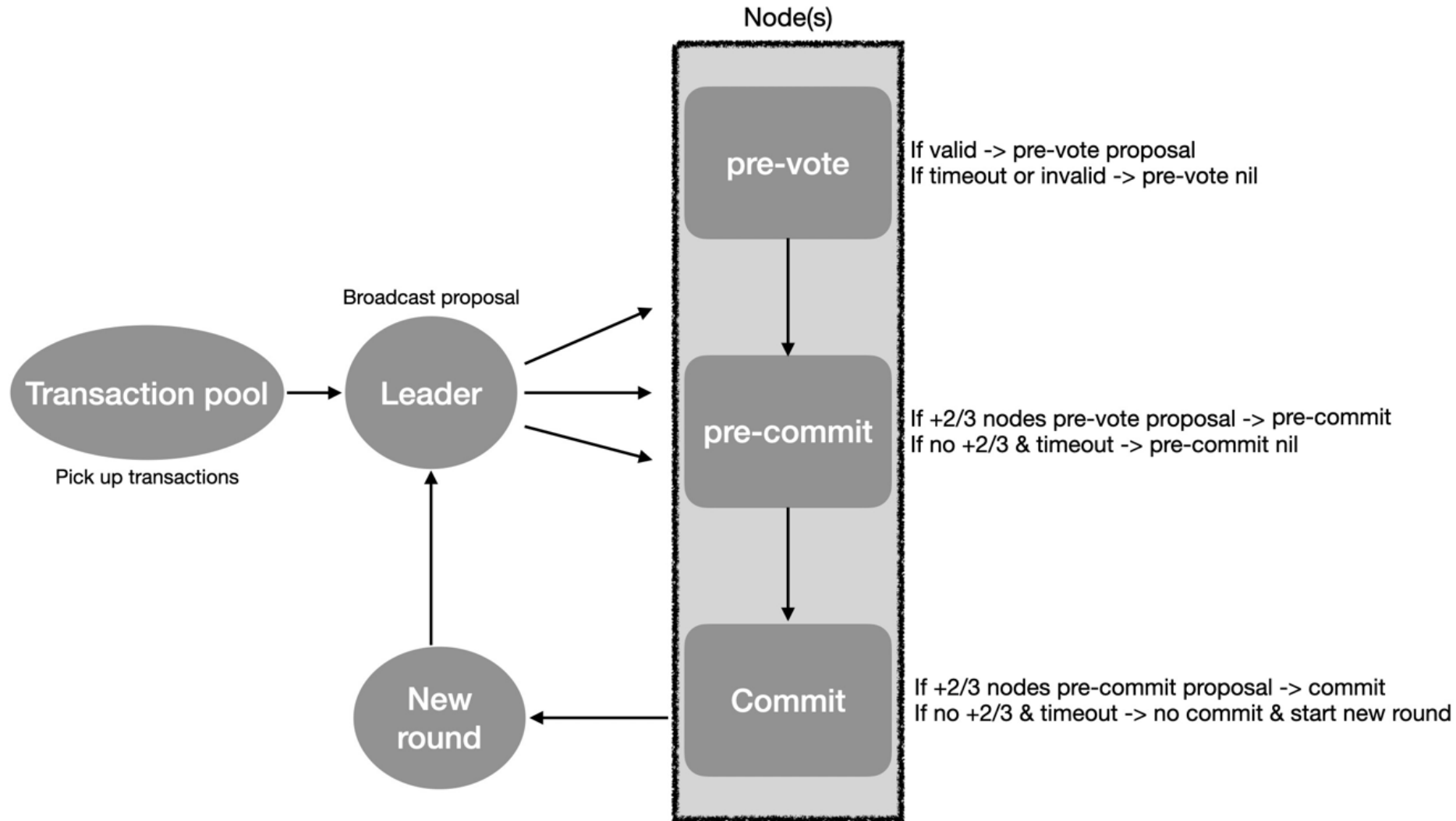


IBFT

23



Tendermint – high-level overview

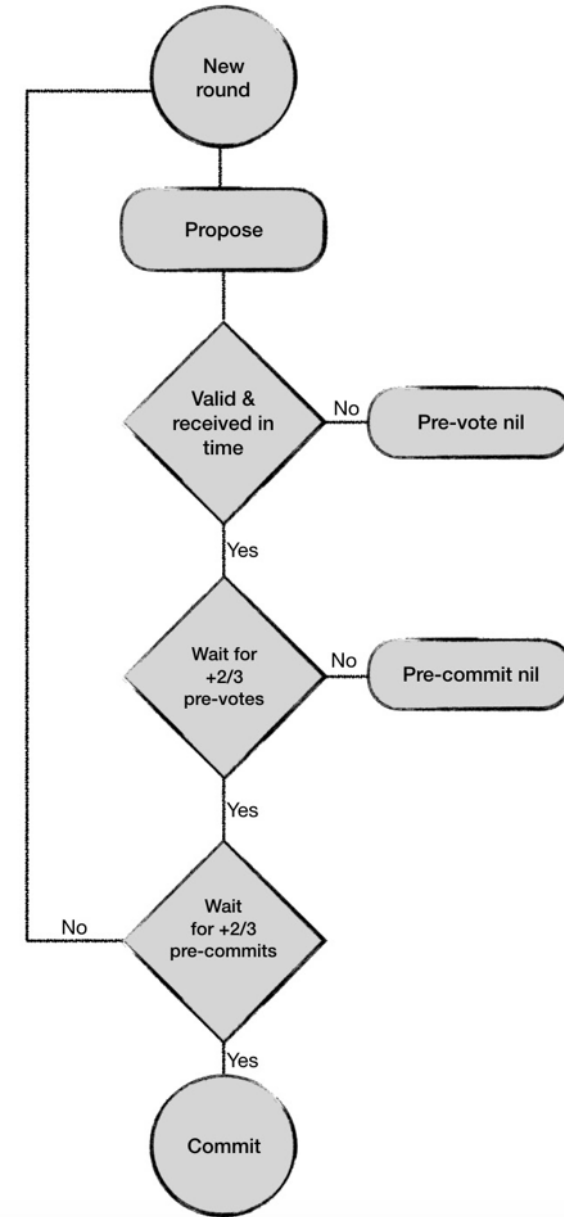


Tendermint process flow

The Tendermint process is simple:

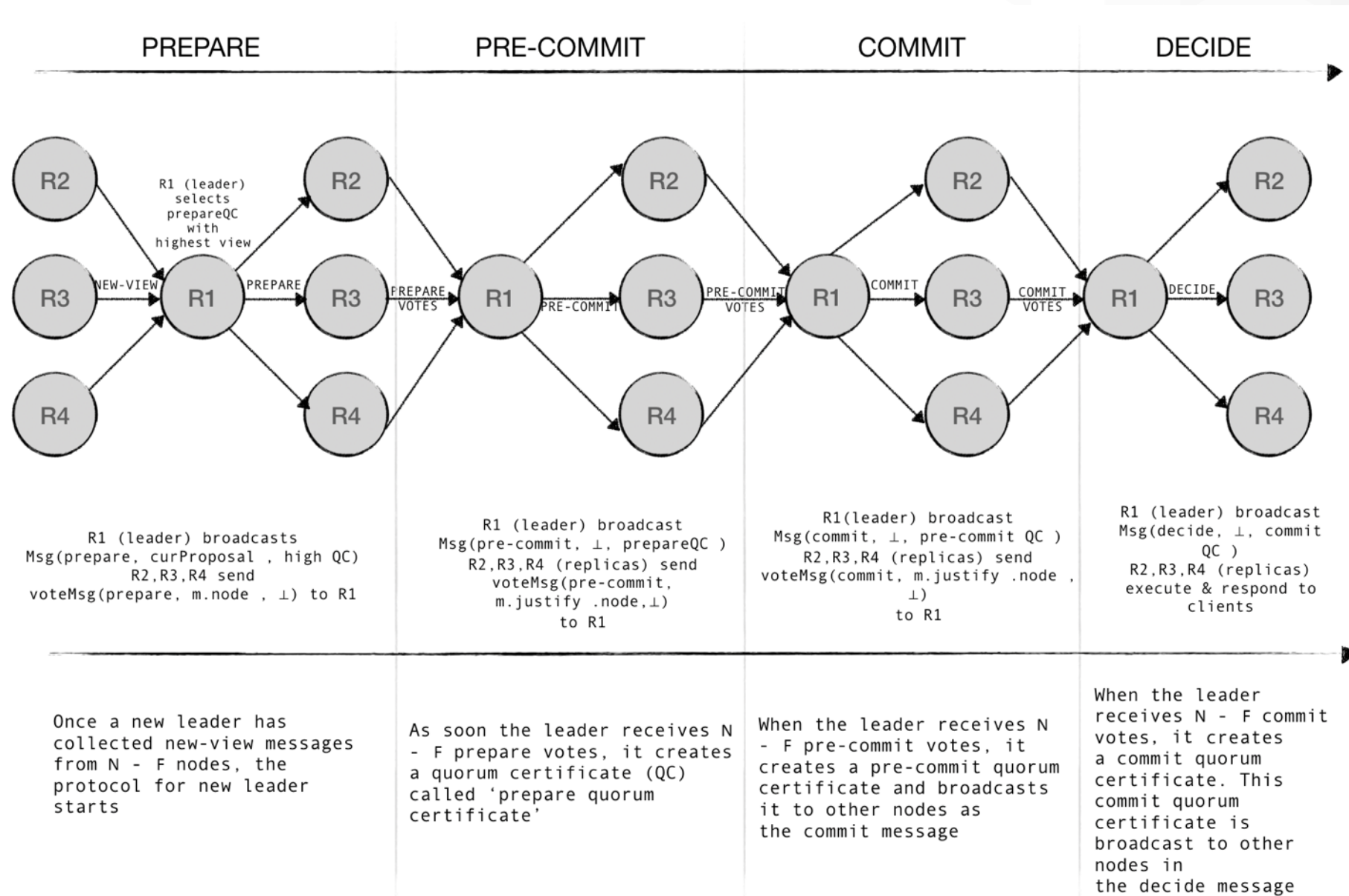
Proposal → *Pre-vote* → *Pre-commit*

We can visualize the protocol flow with the diagram shown here.



HotStuff

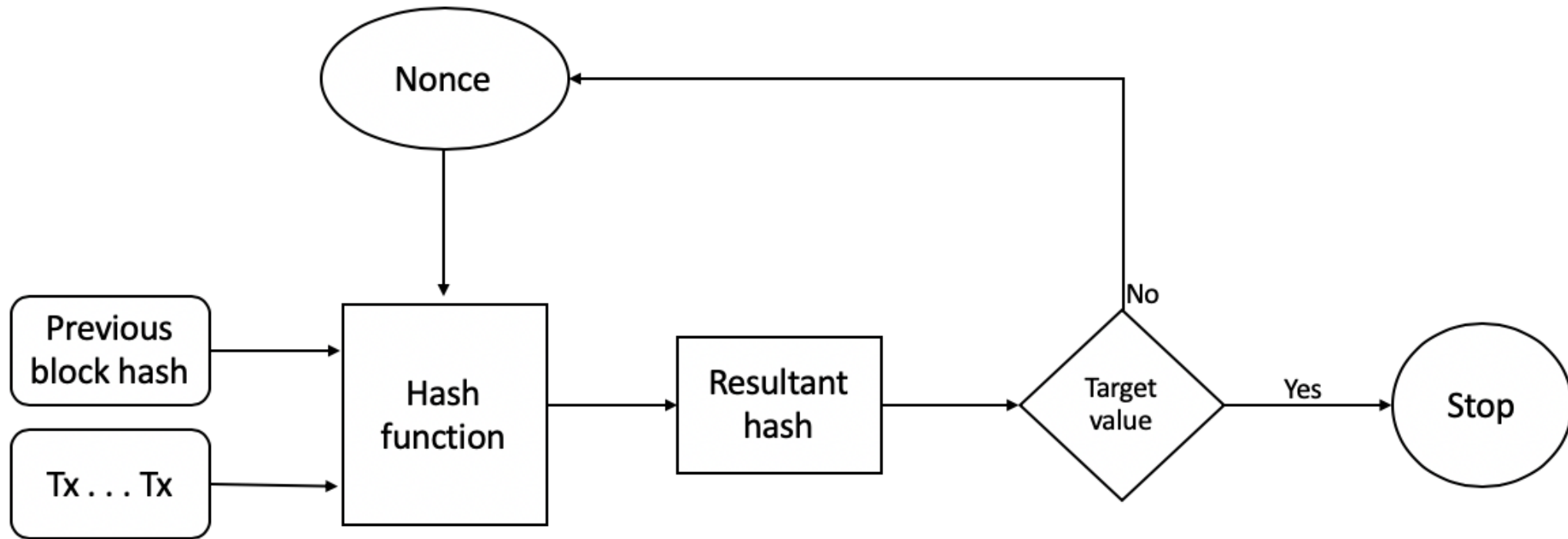
26



Proof of Work

A node that proposes a block has to find a nonce such that:

$H(\text{nonce} \parallel \text{previous hash} \parallel \text{Tx} \parallel \text{Tx} \parallel \dots \parallel \text{Tx}) < \text{Threshold value}.$

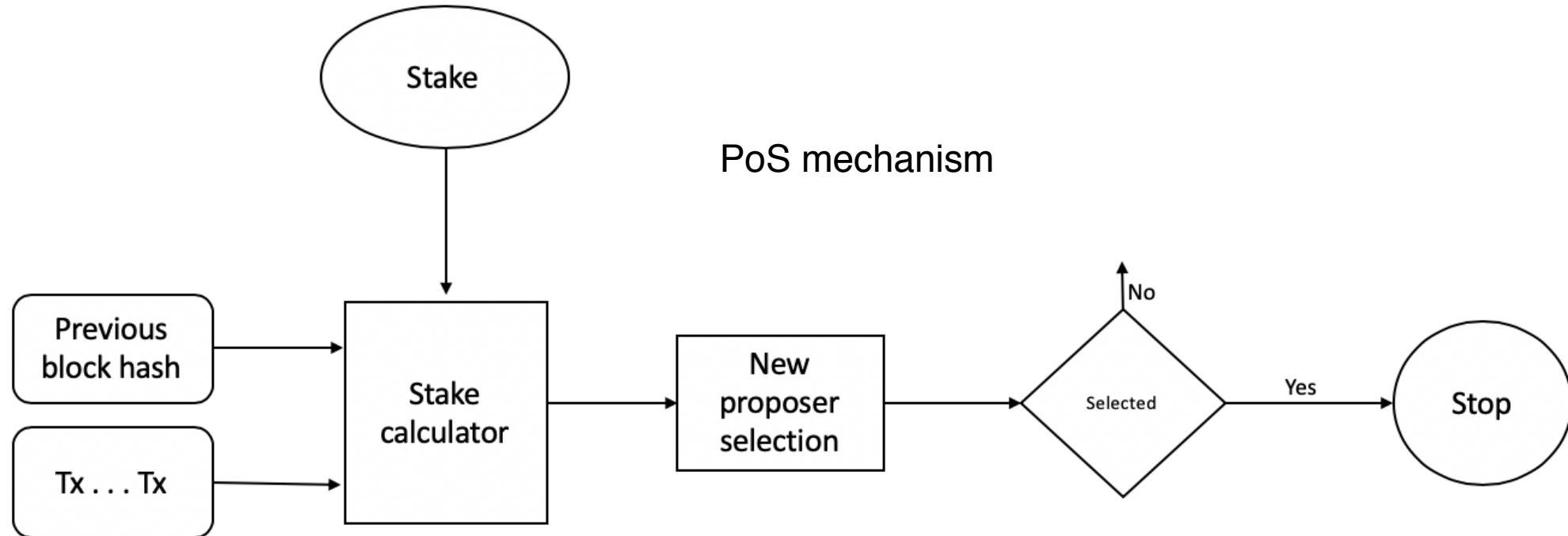


PoS

Types of PoS include:

- Chain-based PoS
- Committee-based PoS
- Delegated PoS

Here, a stake calculator function is used to calculate the amount of staked funds and, based on that, select a new proposer:



Choosing an algorithm

- Finality
- Speed
- Performance
- Scalability

Exercise



- Read the HotStuff research paper at <https://arxiv.org/pdf/1803.05069.pdf> and find out how optimistic responsiveness is achieved.

Summary

In this presentation, we covered:

- Prominent protocols in blockchain and traditional distributed system consensus.
- Proof of Work.
- Proof of Stake.
- Traditional BFT protocols.
- Some more recent protocols, such as HotStuff.